



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E
INDUSTRIAL

CARRERA DE INGENIERÍA EN SOFTWARE

INFORME DE PROYECTO
CREDIT SIMULATOR

INTEGRANTES:

Manobanda Luis

Redroban Joselyn

ASIGNATURA: Metodologías Ágiles

DOCENTE: Ing. Hernán Fabricio Naranjo Ávalos, Mg.

CARRERA: Ingeniería en Software

SEMESTRE: Cuarto - "B"

CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

FECHA: 24/09/2026

Ambato - Ecuador

1. Introducción

Credit Simulator es un proyecto académico orientado a la simulación de préstamos mediante una aplicación web sencilla y accesible. Su propósito es mostrar de forma clara cuánto se pagará en un crédito y cómo cambia el saldo durante el tiempo, considerando los métodos francés y alemán. El usuario puede ingresar, seleccionar un producto financiero, comparar el resultado de la amortización y guardar la simulación para consultarla después.

El desarrollo se organizó con una metodología ágil adaptada al tamaño del equipo. Se dividieron las funcionalidades en historias de usuario, se trabajó por entregas incrementales y se revisó el funcionamiento de cada parte antes de integrarla. Así, el proyecto avanzó desde la autenticación y el catálogo hasta el cálculo, el historial y la exportación de reportes.

2. Descripción del proyecto

2.1 Resumen

Credit Simulator es una aplicación web que permite calcular y visualizar la tabla de amortización de un crédito según su tipo (personal, hipotecario, vehicular, educativo o PyME). El usuario inicia sesión, selecciona el tipo de crédito, ingresa el monto y el plazo en meses, y elige el método de amortización: **francés** (cuota fija) o **alemán** (cuota decreciente). El sistema obtiene la tasa correspondiente, calcula la tabla completa (cuota, interés, abono a capital y saldo por mes), la guarda en la base de datos y permite consultarla y descargarla como reporte.

2.2 Objetivo general

Desarrollar una aplicación web de simulación de créditos que permita calcular tablas de amortización mediante los métodos francés y alemán, guardar los resultados asociados al usuario y exportarlos en formatos de uso común.

2.3 Objetivos específicos

- Implementar registro e inicio de sesión con contraseñas protegidas y tokens JWT.
- Organizar el catálogo de productos con tasas, montos mínimos, montos máximos y plazos máximos.
- Calcular cuotas, intereses, abonos a capital y saldos para los métodos francés y alemán.

- Guardar las simulaciones y permitir al usuario consultar su historial y el detalle de cada resultado.
- Generar reportes de la tabla de amortización en PDF, Excel y CSV.

2.4 Alcance

El sistema incluye registro, autenticación, selección de cinco tipos de crédito (Personal, Hipotecario, Vehicular, Educativo y PyME), validación de rangos, simulación con dos métodos de amortización, almacenamiento del historial, consulta del detalle y exportación de reportes. La interfaz está construida con React y las API están separadas por responsabilidad.

El alcance no incluye aprobación bancaria, desembolso real, conexión con una entidad financiera, manejo de roles administrativos, pagos reales, seguros, comisiones, moras ni cambios de tasas en el tiempo. Las tasas y los límites utilizados corresponden al catálogo de datos inicial del proyecto.

3. Metodología ágil aplicada

3.1 Marco de trabajo

Se utilizó Scrum de forma adaptada. El equipo organizó el trabajo en un product backlog, priorizó las funcionalidades principales y las distribuyó en incrementos cortos. Esta elección permitió entregar primero el acceso y la estructura base, después el cálculo de créditos y finalmente el historial y los reportes, manteniendo una revisión frecuente del resultado.

3.2 Roles del equipo

Tabla 1: Roles del equipo de trabajo

Integrante	Rol	Responsabilidades
Manobanda Luis	Desarrollo y análisis	Diseño de la solución, implementación de servicios y cálculos de amortización, integración con PostgreSQL y documentación.
Redroban Joselyn	Desarrollo y pruebas	Construcción de la interfaz React, integración con las API, revisión de flujos de usuario, pruebas funcionales y apoyo en la documentación.

3.3 Product Backlog

Tabla 2: Historias de usuario del Product Backlog

ID	Historia de usuario	Prioridad	Sprint
HU-01	Como usuario quiero registrarme para acceder a mis simulaciones.	Alta	1
HU-02	Como usuario quiero iniciar sesión para utilizar el simulador.	Alta	1
HU-03	Como usuario quiero elegir un tipo de crédito y conocer sus límites.	Alta	2
HU-04	Como usuario quiero simular con el método francés para obtener una cuota fija.	Alta	2
HU-05	Como usuario quiero simular con el método alemán para observar cuotas decrecientes.	Alta	2
HU-06	Como usuario quiero consultar mi historial y revisar el detalle de una simulación.	Media	3
HU-07	Como usuario quiero descargar la tabla en PDF, Excel o CSV.	Media	3

3.4 Planificación de Sprints

Tabla 3: Resumen de Sprints

Sprint	Fechas	Objetivo y entregable
Sprint 1	Análisis y base	Crear la estructura del proyecto, autenticación, persistencia inicial y configuración de PostgreSQL.
Sprint 2	Catálogo y cálculo	Implementar el catálogo de créditos, las validaciones y los métodos francés y alemán.
Sprint 3	Consulta y entrega	Integrar la interfaz final, historial, detalle, exportaciones, pruebas y documentación.

3.5 Ceremonias y evidencias

La planificación se realizó al inicio de cada incremento para seleccionar las historias prioritarias. Durante el desarrollo se mantuvo comunicación directa para revisar avances, resolver bloqueos y coordinar la integración entre frontend, servicios y base de datos. Al terminar cada incremento se revisó el flujo completo y se identificaron mejoras para el siguiente. La

evidencia principal del trabajo colaborativo queda respaldada por la organización del backlog, la estructura del repositorio y los cambios registrados en Git.

4. Arquitectura del sistema

4.1 Arquitectura de microservicios

El sistema está construido bajo una arquitectura de microservicios. Cada servicio es una aplicación independiente con su propia base de datos, y ningún servicio accede directamente a la base de otro.

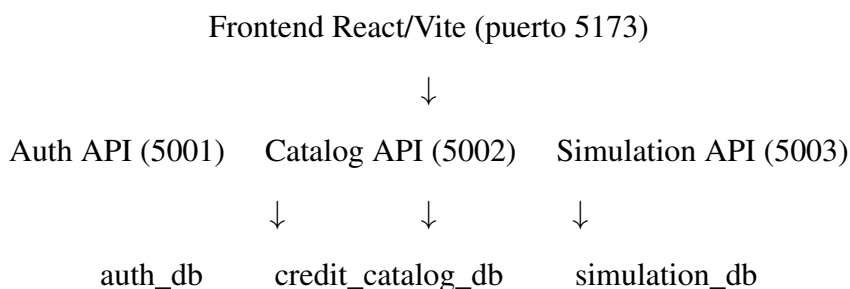
Tabla 4: Microservicios del sistema

Microservicio	Responsabilidad	Base de datos
Auth Service	Login y registro de usuarios	auth_db
Credit Catalog Service	Catálogo de tipos de crédito y sus tasas de interés	credit_catalog_db
Simulation Service	Cálculo de amortización (francés/alemán) e historial	simulation_db

El Simulation Service consulta la tasa de interés al Credit Catalog Service mediante HTTP, respetando el principio de que cada microservicio solo accede a su propia base de datos.

4.2 Diagrama de arquitectura

La solución puede representarse en cuatro niveles. El navegador ejecuta el frontend React/Vite y se comunica con tres API ASP.NET Core. Auth Service administra usuarios y tokens; Credit Catalog Service entrega los productos y sus tasas; Simulation Service calcula y almacena las simulaciones. Cada API utiliza una base independiente en PostgreSQL. Para obtener la tasa, Simulation Service consulta al catálogo mediante HTTP, sin acceder directamente a la base de datos de otro servicio.



4.3 Diagrama de base de datos

La base ‘auth_db’ contiene los usuarios, su correo, el nombre de usuario y el hash de la contraseña. ‘credit_catalog_db’ contiene los tipos de crédito, la tasa anual, los límites de monto, el plazo máximo y el estado activo. ‘simulation_db’ contiene la simulación y sus cuotas. Una simulación guarda el identificador del usuario, el producto, monto, plazo, tasa, método, interés total, total pagado y fecha; cada simulación tiene varias cuotas relacionadas por una clave foránea y eliminadas en cascada.

5. Tecnologías utilizadas

Tabla 5: Stack tecnológico

Capa	Tecnologías
Frontend	React (JavaScript), Vite, React Router, Axios, HTML5 y CSS3
Backend	C# con .NET 10 (ASP.NET Core Web API), Entity Framework Core
Seguridad	BCrypt (hash de contraseñas), JWT (manejo de sesión)
Base de datos	PostgreSQL (tres bases independientes, inicializadas con scripts SQL) y pgAdmin 4
Infraestructura	Docker y Docker Compose
Control de versiones	Git y GitHub

6. Requerimientos

6.1 Requerimientos funcionales

- RF-01: registrar un usuario con nombre, correo y contraseña.
- RF-02: iniciar sesión y conservar la sesión mediante un token JWT.
- RF-03: listar tipos de crédito activos y mostrar sus límites.
- RF-04: validar monto, plazo y método antes de crear una simulación.
- RF-05: calcular una tabla de amortización por el método francés.
- RF-06: calcular una tabla de amortización por el método alemán.
- RF-07: guardar la simulación y consultar el historial del usuario autenticado.

- RF-08: consultar el detalle de una simulación propia.
- RF-09: exportar la simulación en PDF, Excel o CSV.

6.2 Requerimientos no funcionales

- RNF-01: las contraseñas se almacenan mediante BCrypt y las rutas privadas usan JWT.
- RNF-02: cada microservicio mantiene su propia base de datos y expone una API HTTP.
- RNF-03: la interfaz debe mostrar mensajes de validación, carga y error comprensibles.
- RNF-04: la interfaz debe adaptarse a pantallas de escritorio y dispositivos móviles.
- RNF-05: el sistema debe ejecutarse localmente con .NET, Node.js, PostgreSQL o Docker.
- RNF-06: los datos numéricos deben conservar dos decimales y el saldo final debe llegar a cero.

7. Diseño y modelo de amortización

7.1 Método francés

El método francés, también llamado sistema de cuota fija, mantiene constante la cuota periódica durante todo el plazo del préstamo. Cada cuota está formada por una parte de intereses y otra de amortización del capital. Al inicio del préstamo predominan los intereses, porque se calculan sobre un saldo mayor; conforme disminuye el saldo, los intereses bajan y aumenta la parte destinada a amortizar capital.

Con capital inicial P , tasa periódica i y n periodos, la cuota constante C se obtiene mediante la fórmula de una anualidad:

$$C = \frac{P \cdot i}{1 - (1 + i)^{-n}}$$

Para cada periodo k , el interés y la amortización se calculan así:

$$I_k = S_{k-1} \cdot i, \quad A_k = C - I_k, \quad S_k = S_{k-1} - A_k,$$

donde S_{k-1} es el saldo pendiente al inicio del periodo, I_k es el interés, A_k es el abono a capital y S_k es el saldo restante. En la implementación del simulador, estos valores se calculan periodo a periodo y se guardan en la tabla de amortización.

Como ejemplo, para un préstamo de 10000 euros a cuatro años con una tasa anual del 6% y pagos anuales, la tasa periódica es $i = 0,06$ y $n = 4$. La cuota aproximada es:

$$C = \frac{10000 \cdot 0,06}{1 - (1 + 0,06)^{-4}} \approx 2885,91 \text{ euros.}$$

La primera cuota contiene 600,00 euros de intereses y aproximadamente 2285,91 euros de amortización. En los periodos siguientes los intereses disminuyen y el abono a capital aumenta, mientras la cuota permanece fija.

7.2 Método alemán

El método alemán, o sistema de capital constante, amortiza la misma cantidad de capital en cada periodo. Por ello, el saldo disminuye de forma uniforme y los intereses también disminuyen; como resultado, la cuota total es decreciente.

Si P es el capital inicial y n el número de periodos, la amortización constante de capital es:

$$A = \frac{P}{n}, \quad C_k = A + S_{k-1} \cdot i$$

donde A es el abono constante a capital, S_{k-1} es el saldo del periodo anterior, i es la tasa periódica y C_k es la cuota total del periodo k . El interés y el saldo se actualizan con:

$$I_k = S_{k-1} \cdot i, \quad S_k = S_{k-1} - A.$$

Por ejemplo, para un capital de 10000 euros a cuatro periodos, la amortización de capital es $A = 2500$ euros por periodo. Con una tasa del 6%, la primera cuota es 3100 euros ($2500 + 600$), la segunda es 2950 euros, la tercera es 2800 euros y la cuarta es 2650 euros. La cuota baja porque los intereses se calculan cada vez sobre un saldo menor, aunque el abono a capital permanece constante.

7.3 Comparación de los métodos

El método francés facilita la planificación porque la cuota es constante, pero concentra una mayor proporción de intereses al comienzo. El método alemán reduce más rápidamente el saldo y genera un menor pago total de intereses en igualdad de capital, tasa y plazo; sin embargo, exige una cuota inicial más alta. El simulador permite seleccionar ambos métodos y presenta para cada periodo la cuota, el interés, el abono a capital y el saldo pendiente.

7.4 Casos de prueba

Tabla 6: Casos de prueba funcionales

ID	Acción	Resultado esperado	Estado
CP-01	Registrar un usuario nuevo y volver a la aplicación.	Se crea la cuenta y se recibe un token válido.	Conforme
CP-02	Iniciar sesión con datos correctos e incorrectos.	Se permite el acceso correcto y se rechazan credenciales inválidas.	Conforme
CP-03	Solicitar el catálogo de créditos.	Se muestran los cinco productos activos con tasa y límites.	Conforme
CP-04	Simular 10000 a 12 meses con método francés.	Se genera una cuota fija aproximada de 888,49 con tasa anual del 12%.	Conforme
CP-05	Simular 10000 a 12 meses con método alemán.	La primera cuota es aproximadamente 933,33 y la última 841,67.	Conforme
CP-06	Enviar un monto o plazo fuera de los límites.	La API devuelve un mensaje de validación y no guarda datos.	Conforme
CP-07	Consultar historial y detalle.	Solo aparecen las simulaciones asociadas al usuario autenticado.	Conforme
CP-08	Exportar una simulación en PDF, XLSX y CSV.	Se descarga un archivo con datos y cuotas de la simulación.	Conforme

8. Implementación

8.1 Estructura del repositorio

El repositorio se divide en tres áreas principales. ‘frontend/credit-web’ contiene la aplicación React, las páginas de login, registro, simulador, historial y detalle, además de los servicios HTTP del navegador. ‘services’ contiene ‘auth-service’, ‘credit-catalog-service’ y ‘simulation-service’, cada uno con su proyecto .NET, controladores, modelos y contexto de Entity Framework. ‘infrastructure’ contiene Docker Compose, la configuración de PostgreSQL y los scripts que crean las tres bases de datos.

8.2 Configuración local

Para ejecutar el proyecto se instala .NET SDK 10, Node.js 22 o superior y Docker Desktop. Desde la raíz se puede iniciar PostgreSQL con `docker compose -f infrastructure/docker-compose.yml up -d postgres`. En un volumen nuevo, los scripts de `'infrastructure/postgres/init'` crean `'auth_db'`, `'credit_catalog_db'` y `'simulation_db'` y cargan el catálogo.

Después se restauran y ejecutan las tres API en los puertos 5001, 5002 y 5003. En otra terminal se ejecuta `'npm install'` dentro de `'frontend/credit-web'` y `'npm run dev'`. Finalmente se abre `'http://localhost:5173'`. Como alternativa, `'docker compose -f infrastructure/docker-compose.yml up --build'` levanta toda la solución, incluido pgAdmin en el puerto 5050.

8.3 Capturas de pantalla

La interfaz contiene las vistas de inicio de sesión y registro, el simulador con selección de producto y método, la tabla de amortización con sus totales, el historial ordenado por fecha y el detalle de cada simulación. En el resultado también se muestran los botones para descargar PDF, XLSX y CSV. La captura de la página principal se incorpora en el anexo como evidencia visual.

9. Pruebas

Las pruebas se organizaron por capas. En Auth Service se verificaron el registro, el inicio de sesión, el rechazo de credenciales incorrectas y la prevención de usuarios duplicados. En Credit Catalog Service se verificó la consulta de productos activos y la respuesta cuando un identificador no existe. En Simulation Service se probaron los dos calculadores, los límites del catálogo, la autorización, el historial y las exportaciones.

También se realizó una prueba de punta a punta: registrar un usuario, iniciar sesión, crear una simulación, revisar la tabla, abrir el historial, consultar el detalle y descargar los tres formatos. La validación numérica se contrastó con el caso de 10000, 12 meses y tasa anual del 12 % documentado en el README del proyecto.

10. Trabajo colaborativo y control de versiones

El trabajo se organizó separando responsabilidades sin perder la revisión conjunta. Manobanda Luis se enfocó en la estructura de los servicios, la persistencia y los calculadores; Redroban Joselyn se enfocó en las pantallas, los formularios y la integración del frontend. Ambos

revisaron los flujos completos y la documentación antes de cerrar cada incremento.

Git se utilizó para conservar el historial de cambios y facilitar la integración. La distribución por carpetas del repositorio permite identificar qué parte corresponde al frontend, a cada API y a la infraestructura, mientras que los mensajes de commit sirven como evidencia del avance.

11. Retrospectiva y lecciones aprendidas

Funcionó bien separar la solución por servicios, porque cada responsabilidad quedó más clara y fue posible probarla de manera independiente. También fue útil validar el catálogo antes del cálculo, ya que de esa forma la tasa y los límites usados en la tabla provienen de una fuente única.

Como aprendizaje, la integración entre frontend, API y bases de datos debe probarse desde temprano, no solamente cuando cada parte parece terminada. En una siguiente iteración convendría agregar pruebas automatizadas para los calculadores y los controladores, centralizar secretos fuera de los archivos de desarrollo y añadir roles administrativos para gestionar el catálogo. La comunicación frecuente y la entrega incremental ayudaron a detectar estas mejoras antes del cierre.

12. Conclusiones y recomendaciones

12.1 Conclusiones

Credit Simulator cumple el objetivo de ofrecer una forma clara de comparar dos sistemas de amortización. El método francés permite observar una cuota fija, mientras el método alemán muestra una cuota inicial mayor que disminuye con el saldo. La tabla presenta el detalle de cada periodo y el sistema conserva el resultado para futuras consultas.

La arquitectura con React, tres API ASP.NET Core y PostgreSQL separado por servicio favorece la organización del código y la independencia de los datos. La autenticación, las validaciones, el historial y las exportaciones convierten el cálculo matemático en un flujo de uso completo para el usuario.

12.2 Recomendaciones

Como trabajo futuro se recomienda incorporar pruebas unitarias y de integración automatizadas, configurar un proveedor seguro para las claves y contraseñas, añadir manejo de roles y permitir la administración del catálogo. También se pueden incluir seguros, comisiones, pagos anticipados, tasas variables y una comparación visual de los costos totales. Para producción

sería conveniente usar HTTPS, observabilidad, copias de seguridad y una política de datos más estricta.

Referencias Bibliográficas

- [1] Edufinet. “¿Cómo se amortiza un préstamo?”. Disponible en: <https://edufinet.com/creditos-y-prestamos/como-se-amortiza-un-prestamo/>
- [2] Wikipedia. “Préstamo francés”. Disponible en: https://es.wikipedia.org/wiki/Préstamo_francés
- [3] Universitat de Barcelona. “Apuntes MOF OMADO”. Disponible en: <https://diposit.ub.edu/dspace/bitstream/2445/143881/5/2019%20Apuntes%20MOF%20OMADO.pdf>
- [4] INEAF. “Préstamo alemán o el sistema de amortización alemán: todos los detalles”. Disponible en: <https://www.ineaf.es/tribuna/prestamo-aleman/>
- [5] Fundación Mapfre. “Sistema alemán de amortización”. Disponible en: <https://segurosypensionesparatodos.fundacionmapfre.org/glosario/sistema-aleman-de-amortizacion/>
- [6] BIESS. “Glosario”. Disponible en: <https://www.biess.fin.ec/servicio-al-cliente/glosario?l=T>

A. Anexos



Figura 1: Página web principal de Credit Simulator.